

SIMATS ENGINEERING

Assignment

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN

COURSE CODE: CSA1107

COURSE OUTCOME COVERED

CO1: Analyse the fundamentals of Object-Oriented Systems Development, including object basics, Object-Oriented Systems Development Life Cycle, Object-Oriented Methodologies, Model-Driven Development (MDD), and Agile practices.

CO2: Apply UML techniques including Use Case, Class, Interaction, State Transition, Activity, Package, Component, and Deployment Diagrams to model a software system.

CO3: Analyse objects, classes, attributes, methods, and relationships through Use Case Modelling, Object Analysis, Classification, and identification of object relationships.

Assignment Weightage: 100%

QUESTIONS: 1

Total Marks:100

Annexure A

Assignment

Code of Conduct:

I Sanjana.G(192511020) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Sanjana.G

Signature of the student:

Course Code & Name**CSA1107 – Object Oriented Analysis and Design**

Assignment Title	Object-Oriented Analysis and UML-Based Design of an Intelligent Food Waste Reduction and Redistribution System (IFWRRS)
Course Outcome(s) – CO	CO1: Analyse Object-Oriented Systems Development concepts and methodologies to understand and model a real-world software problem. CO2: Apply UML modelling techniques including Use Case, Class, Interaction, State Transition, Activity, Package, Component, and Deployment Diagrams. CO3: Analyse objects, classes, attributes, methods, and relationships through Use Case Modelling, Object Analysis, Classification, and relationship identification.
Bloom's Taxonomy Level	BL3 – Apply, BL4 – Analyse, BL5 – Evaluate, BL6 – Create
SDG Mapping (SDG 1–17)	SDG 2 – Zero Hunger SDG 11 – Sustainable Cities and Communities SDG 12 – Responsible Consumption and Production SDG 13 – Climate Action

Problem Statement

Analyse and develop an Object-Oriented Analysis and Design model for an Intelligent Food Waste Reduction and Redistribution System (IFWRRS) that connects restaurants, hotels, supermarkets, food suppliers, charitable organizations, delivery partners, and administrators to reduce avoidable food waste and facilitate timely redistribution of surplus food.

The system shall allow registered food providers to record surplus food, specify quantity, type, preparation time, expiry period, storage requirements, and pickup location. The system shall identify suitable recipient organizations based on food requirements, location, quantity, urgency, and availability.

Recipient organizations shall be able to view available food donations, request suitable items, confirm collection, and update distribution status. Delivery partners shall receive pickup and delivery assignments and update the transportation status. Administrators shall monitor food donations, organizations, deliveries, users, and system reports.

The system shall generate notifications when surplus food is approaching its expiry period and support intelligent matching between food providers and recipient organizations. Working from this scenario, students shall identify actors, objects, classes, attributes, methods, and relationships and develop Use Case, Class, Sequence, Activity, State Transition, Package, Component, and Deployment Diagrams.

Students shall analyse object responsibilities and apply suitable design principles and design patterns to achieve high cohesion, low coupling, reusability, maintainability, and scalability.

Constraints

The design shall clearly distinguish different stakeholders and their responsibilities. All identified classes, attributes, operations, and relationships shall be traceable to the identified use cases.

The system shall consider food expiry, notification timing, matching criteria, delivery status, user roles, data consistency, and transaction management. UML diagrams shall be mutually consistent and shall accurately represent the proposed system behaviour.

The final submission shall include assumptions, constraints, object responsibility analysis, design-pattern justification, and traceability between requirements, use cases, and design elements.

Tools Can Be Used

- StarUML
- Visual Paradigm
- Draw.io
- PlantUML
- Papyrus Tools
- ArgoUML
- Enterprise Architect
- Lucidchart

Assessment Rubrics

Criteria (CO Mapping)	Max Marks	Excellent	Good	Satisfactory	Needs Improvement
Problem Understanding, Object Identification & Requirements Analysis (CO1)	10	Clearly analyses the problem domain and accurately identifies all major actors, objects, system responsibilities, and requirements from the given scenario.	Identifies most actors, objects, and requirements with minor gaps or inaccuracies.	Identifies basic actors and objects but analysis is incomplete or contains some inconsistencies.	Poor understanding of the problem; major actors, objects, and requirements are missing or incorrectly identified.
Use Case Modelling & Actor Identification (CO2)	10	Correctly identifies all relevant actors and use cases; Use Case Diagram clearly represents system boundary, relationships, and interactions.	Most actors and use cases are correctly identified with minor modelling errors.	Basic actors and use cases are identified, but relationships or system boundary need improvement.	Actors and use cases are poorly identified; diagram is incomplete or inconsistent.
Class Identification, Attributes, Methods &	15	Classes, attributes, methods, associations, aggregation/composition, inheritance, and	Class structure is mostly correct with minor errors in	Basic classes and relationships are identified	Classes, attributes, methods, or relationships

Criteria (CO Mapping)	Max Marks	Excellent	Good	Satisfactory	Needs Improvement
Relationships (CO2/CO3)		dependencies are accurately identified and logically justified.	attributes, methods, or relationships.	but lack detail or contain several modelling issues.	are missing, inappropriate, or poorly structured.
UML Behavioural & Interaction Modelling (CO2)	15	Sequence, Activity, and State Transition Diagrams accurately represent system behaviour and are fully consistent with the identified use cases and classes.	Behavioural diagrams are mostly correct with minor inconsistencies.	Basic behavioural diagrams are provided but some interactions, activities, or states are incomplete.	Behavioural diagrams are missing, unclear, or inconsistent with the proposed system.
UML Architectural Modelling & Design Evaluation (CO3)	15	Package, Component, and Deployment Diagrams clearly represent modular structure, system components, dependencies, and deployment environment with strong justification.	Architectural diagrams are mostly correct with minor omissions or modelling errors.	Basic architectural representation is provided but lacks clarity or sufficient justification.	Architectural diagrams are incomplete, inappropriate, or inconsistent with the system design.
Object Responsibilities, Design Principles, Axioms & Patterns (CO3)	15	Responsibilities are appropriately assigned; design axioms, cohesion, coupling, encapsulation, abstraction, and suitable design patterns are clearly justified and effectively applied.	Design principles and patterns are appropriately applied with minor limitations in justification.	Some design principles are identified, but application or justification is limited.	Design principles, axioms, responsibilities, or patterns are absent or incorrectly applied.
Consistency, Traceability & Overall Design Quality (CO3)	10	All UML models are mutually consistent, requirements are traceable to use cases/classes, and the overall design is complete, feasible, scalable, and maintainable.	Design is largely consistent and traceable with minor gaps.	Some consistency and traceability are demonstrated, but important links are missing.	Major inconsistencies exist between requirements, use cases, classes, and UML diagrams.
Reflection, SDG Relevance &	10	Provides insightful reflection on design	Provides good reflection with	Reflection is present but	Reflection is missing,

Criteria (CO Mapping)	Max Marks	Excellent	Good	Satisfactory	Needs Improvement
Learning Outcomes		decisions, challenges, OOAD concepts learned, and clearly establishes the relevance of the selected SDGs.	reasonable connection to design decisions and SDGs.	generic with limited discussion of learning and SDG relevance.	superficial, or unrelated to the assignment and SDGs.
Total	100				

Deliverables

To receive full credit, each submission must include the following:

1. **Problem Analysis and Requirement Summary**
2. **Actor Identification**
3. **Use Case Descriptions**
4. **Use Case Diagram**
5. **Object and Class Identification**
6. **Class Diagram**
7. **Sequence/Interaction Diagram(s)**
8. **Activity Diagram**
9. **State Transition Diagram**
10. **Package Diagram**
11. **Component Diagram**
12. **Deployment Diagram**
13. **Object Responsibility Analysis**
14. **Design Axioms/Principles and Design Pattern Justification**
15. **Assumptions and Constraints**
16. **Requirement-to-Use Case/Class Traceability**
17. **Implementation/Prototype and Results**
18. **Reflection on Design Decisions and Learning Outcomes**
19. **GitHub Upload**

Problem Analysis and Requirement Summary

1. Actor Identification

The main actors involved in the Intelligent Food Waste Reduction and Redistribution System (IFWRRS) are:

Actor	Responsibilities
Food Provider	Registers surplus food and provides food details.
Recipient Organization	Searches, requests, collects, and distributes donated food.
Delivery Partner	Performs pickup and delivery and updates delivery status.
Administrator	Manages users, donations, organizations, and deliveries.

These actors represent the major stakeholders of the proposed system.

2. Use Case Descriptions

UC1 – Register Surplus Food

Actor: Food Provider

Description: Enables the food provider to register available surplus food.

Precondition: Food provider must be registered and logged in.

Main Flow:

1. Food provider logs into the system.
2. Selects the Register Surplus Food option.
3. Enters food details.
4. Enters quantity and expiry information.
5. Enters pickup location.
6. Submits the information.
7. System stores the donation information.

Postcondition: Surplus food is available for matching.

UC2 – Find Suitable Recipient

Actor: System / Food Provider

Description: The system finds suitable recipient organizations based on donation and requirement information.

Main Flow:

1. System receives donation details.
2. Identifies recipient requirements.
3. Compares donation and recipient location.
4. Compares required quantity.
5. Compares urgency and availability.
6. Generates suitable matches.

Postcondition: Suitable recipient organizations are identified.

UC3 – Request Food Donation

Actor: Recipient Organization

Description: Enables the organization to request suitable available food donations.

Main Flow:

1. Organization logs into the system.
2. Views available donations.
3. Selects required food.
4. Submits donation request.
5. System records the request.
6. Food provider/administrator processes the request.

UC4 – Manage Delivery

Actor: Delivery Partner

Description: Handles the delivery of donated food.

Main Flow:

1. Delivery partner receives assignment.
2. Travels to pickup location.
3. Picks up food.
4. Updates pickup status.
5. Delivers food to the organization.

6. Updates delivery status.

UC5 – Monitor Expiry

Actor: System / Administrator

Description: Monitors the expiry period of surplus food and generates notifications.

Main Flow:

1. System monitors expiry period.
2. Identifies food nearing expiry.
3. Generates notification.
4. Notification is received by relevant users.
5. Urgent donations can be prioritized.

3. Use Case Diagram

The Use Case Diagram should contain the following actors and use cases:

Food Provider

- Register/Login
- Register Surplus Food
- Update Food Details
- View Donation Status

Recipient Organization

- Register/Login
- View Available Donations
- Request Food
- Confirm Collection
- Update Distribution Status

Delivery Partner

- Login
- View Delivery Assignment
- Pickup Food

- Deliver Food
- Update Delivery Status

Administrator

- Manage Users
- Manage Donations
- Manage Organizations
- Monitor Deliveries
- Generate Reports

System

- Match Food with Recipient
- Monitor Expiry
- Send Notifications

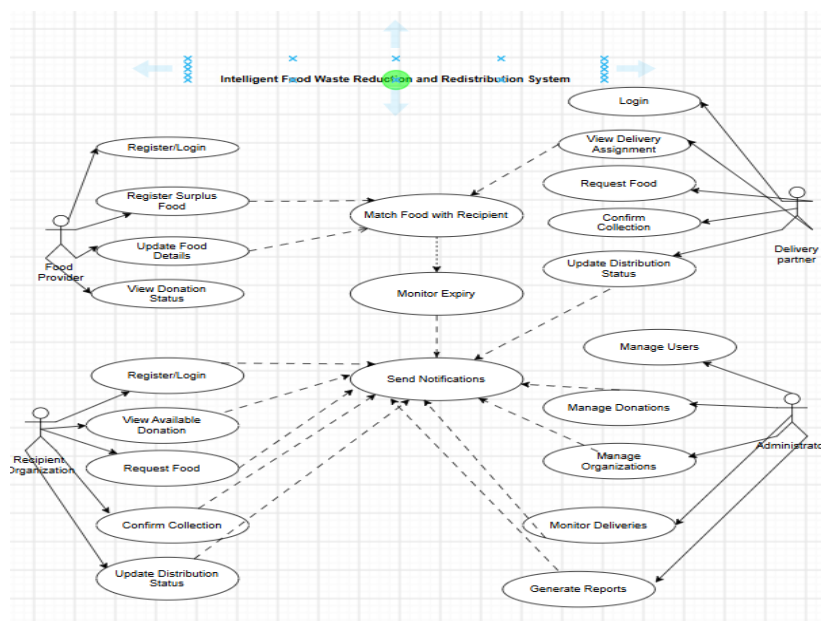


Figure 1. Use Case Diagram of the Food Donation Management System.

4. Object and Class Identification

The major classes identified for the system are:

1. User
2. FoodProvider
3. RecipientOrganization
4. DeliveryPartner
5. Administrator
6. FoodDonation
7. FoodItem
8. FoodRequirement
9. DonationRequest
10. Delivery
11. Notification
12. Location
13. Report

Important Attributes

FoodDonation

- donationId
- foodType
- quantity
- preparationTime
- expiryTime
- storageRequirement
- pickupLocation
- status

Delivery

- deliveryId
- pickupLocation

- deliveryLocation
- pickupTime
- deliveryTime
- deliveryStatus

DonationRequest

- requestId
- requestDate
- quantityRequested
- requestStatus

5. Class Diagram

The main relationships between classes are:

- User is the parent class of FoodProvider, RecipientOrganization, DeliveryPartner, and Administrator.
- FoodProvider creates FoodDonation.
- FoodDonation contains FoodItem.
- RecipientOrganization creates DonationRequest.
- DonationRequest is associated with FoodDonation.
- DeliveryPartner handles Delivery.
- FoodDonation is associated with Delivery.
- FoodDonation generates Notification.
- RecipientOrganization has FoodRequirement.
- FoodProvider and RecipientOrganization are associated with Location.

Example Methods

FoodProvider

- addDonation()
- updateDonation()
- viewDonationStatus()

RecipientOrganization

- viewDonations()
- requestDonation()
- confirmCollection()
- updateDistribution()

DeliveryPartner

- acceptDelivery()
- pickupFood()
- deliverFood()
- updateDeliveryStatus()

Administrator

- manageUsers()
- monitorDonations()
- monitorDeliveries()
- generateReport()

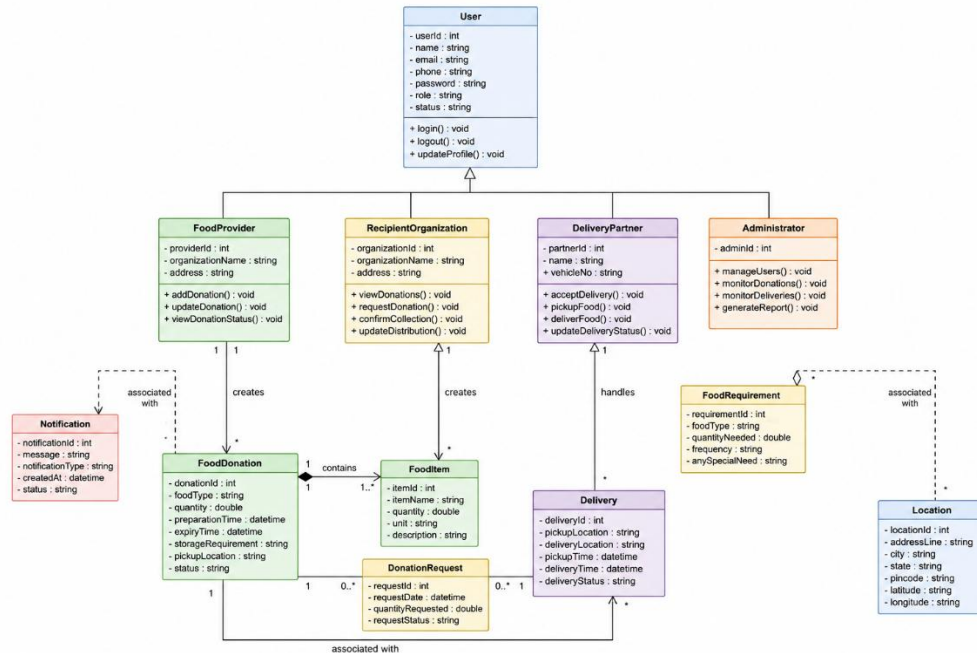


Figure 2. Class Diagram of the Food Donation Management System.

6. Sequence Diagram(s)

Food Donation and Redistribution Sequence

1. Food Provider logs into the system.
2. Food Provider enters surplus food details.
3. Food Provider submits donation.
4. System validates the donation.
5. System searches for recipient organizations.
6. System matches a suitable recipient.
7. System sends donation notification.
8. Recipient Organization views the donation.
9. Recipient Organization submits a request.
10. Recipient Organization confirms the request.
11. System assigns delivery.
12. Delivery Partner confirms pickup.
13. Delivery Partner updates delivery status.
14. Delivery Partner confirms delivery.
15. System notifies the Recipient Organization.
16. Recipient Organization updates distribution status.

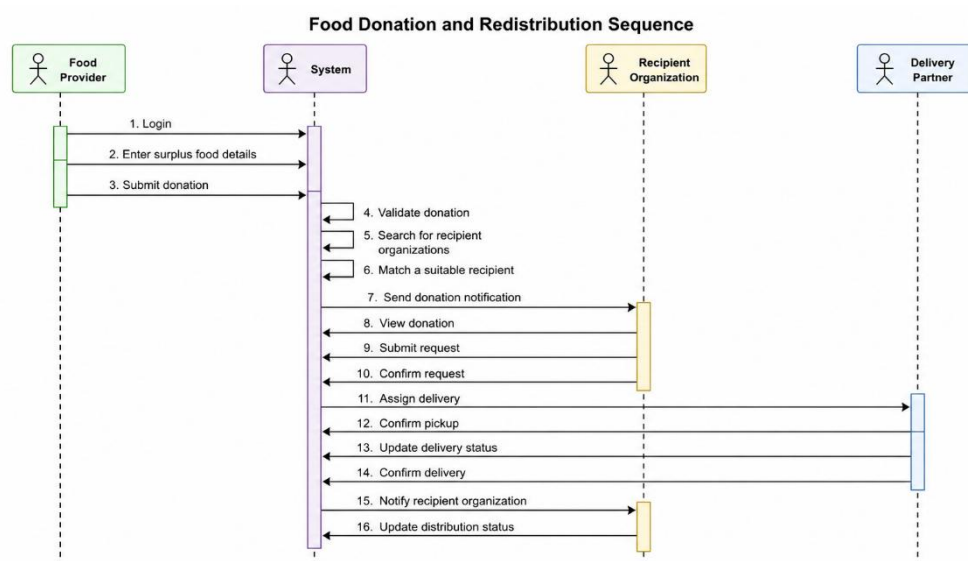


Figure 3. Sequence Diagram of the Food Donation Management System.

7. Activity Diagram

The activity flow of the system is:

Start



Food Provider Login



Enter Surplus Food Details



Validate Information



Register Donation



Check Recipient Requirements



Find Suitable Recipient



Send Notification



Recipient Views Donation



Request Donation



Assign Delivery Partner



Pickup Food



Transport Food



Deliver Food



Recipient Confirms Collection



Update Distribution Status

↓

End

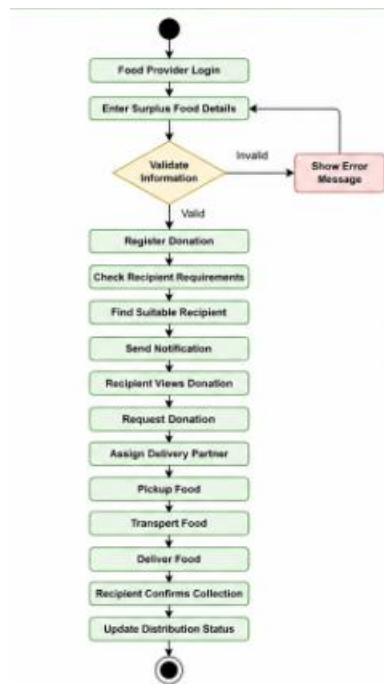


Figure 4.. Activity Diagram of the Food Donation Management System.

8. State Transition Diagram

The **Food Donation** passes through the following states:

Registered

↓

Available

↓

Matched

↓

Requested

↓

Approved

↓

Pickup Assigned

↓

Picked Up

↓

In Transit

↓

Delivered

↓

Distributed

An alternative transition is:

Available → Expired

The system monitors the expiry period and generates notifications when food is approaching expiry.

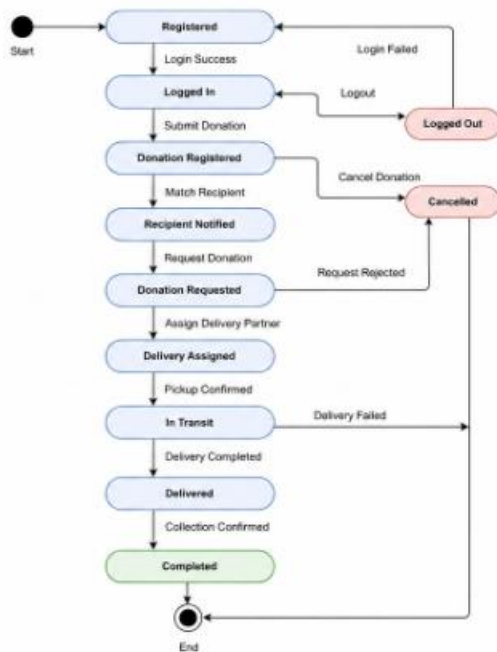


Figure 5. State Transition Diagram of the Food Donation Management System.

9. Package Diagram

The system can be divided into the following packages:

1. User Management

- User
- FoodProvider
- RecipientOrganization
- DeliveryPartner
- Administrator

2. Donation Management

- FoodDonation
- FoodItem
- FoodRequirement

3. Matching Management

- Matching

- MatchingCriteria

4. Request Management

- DonationRequest

5. Delivery Management

- Delivery
- DeliveryAssignment

6. Notification Management

- Notification
- ExpiryNotification

7. Reporting

- Report
- ReportGenerator

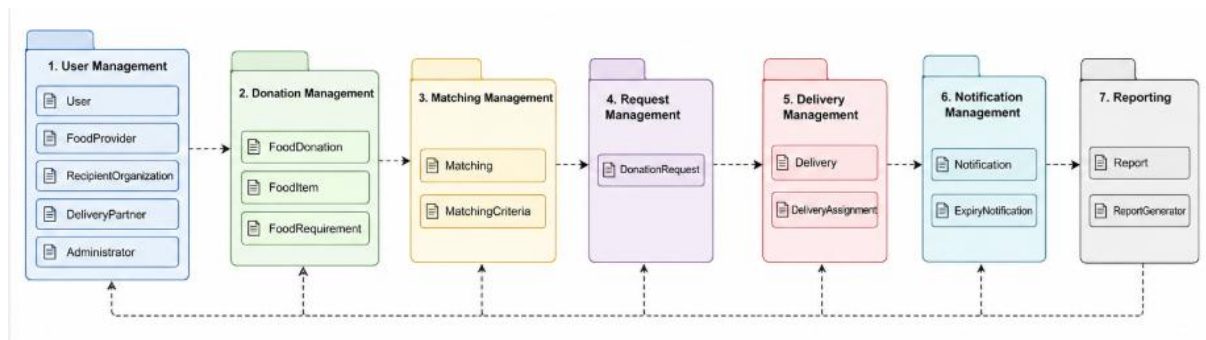


Figure 6.Package Diagram of the Food Donation Management System.

10. Component Diagram

The major components are:

- User Interface
- Authentication Component
- User Management Component
- Donation Management Component
- Food Matching Component
- Request Management Component

- Delivery Management Component
- Notification Component
- Reporting Component
- Database

Component Flow

User Interface → Application Services → Business Components → Database

The Food Matching Component communicates with donation and recipient requirement information. The Notification Component handles expiry-related alerts.

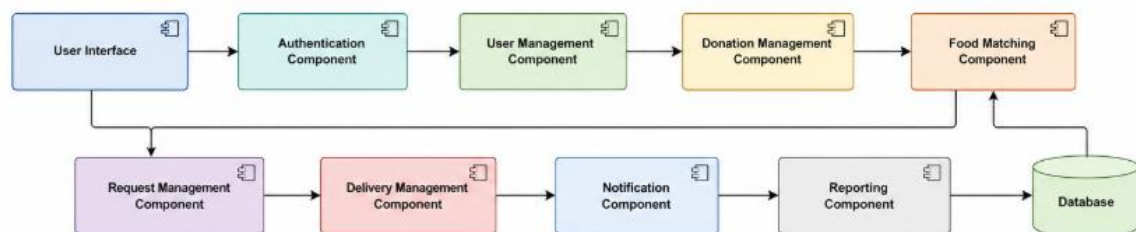


Figure 7. Component Diagram of the Food Donation Management System.

11. Deployment Diagram

Client Devices

- Food Provider Mobile/Web Application
- Recipient Organization Application
- Delivery Partner Application
- Administrator Web Dashboard

↓

Application Server

- Authentication Server
- Donation Management
- Matching Service
- Delivery Management
- Notification Service

- Reporting Service



Database Server

- User Database
- Donation Database
- Organization Database
- Delivery Database
- Transaction Database

The deployment diagram represents communication between client devices, application services, and the database server.

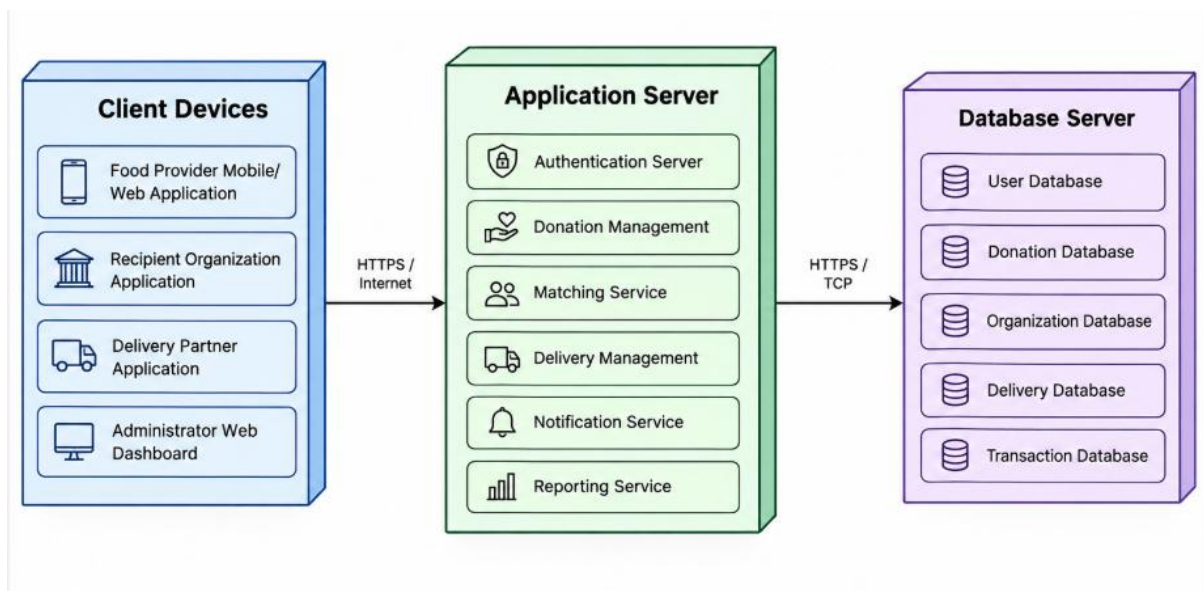


Figure 8. Deployment Diagram of the Food Donation Management System.

12. Object Responsibility Analysis

Object/Class	Main Responsibility
User	Store common user information and authentication details.
FoodProvider	Register and manage surplus food.
RecipientOrganization	Manage food requirements and donation requests.
DeliveryPartner	Manage food pickup and delivery.
Administrator	Monitor and manage the overall system.
FoodDonation	Store surplus food information and donation status.
DonationRequest	Manage requests made by recipient organizations.
Delivery	Track pickup, transportation, and delivery.
Notification	Generate and manage system notifications.
FoodRequirement	Store recipient food requirements.
Report	Provide system monitoring and analytical information.

13. Design Axioms/Principles and Design Pattern Justification

Design Principles

Encapsulation:

Each class keeps its data private and provides methods to access or modify the data.

Abstraction:

Only necessary information is exposed to other components.

High Cohesion:

Each class has a clearly defined responsibility.

Low Coupling:

Classes should have minimum dependency on other classes.

Reusability:

Common functionality should be designed for reuse.

Maintainability:

The system should be divided into independent modules so that future modifications are easier.

Design Patterns

Factory Pattern:

Used to create different types of users such as Food Provider, Recipient Organization, Delivery Partner, and Administrator.

Observer Pattern:

Used for notifications. When a donation approaches expiry or its status changes, relevant users can automatically receive notifications.

Strategy Pattern:

Used for different food-matching strategies based on location, quantity, urgency, or food requirements.

14. Assumptions and Constraints

Assumptions

- Users have valid registration details.
- Food providers provide accurate food and expiry information.
- Recipient organizations provide accurate food requirements.
- Delivery partners have access to required delivery information.
- Users have access to the internet.
- Food providers and recipient organizations follow appropriate food safety practices.
- The system has access to accurate location information.

Constraints

- Food must be redistributed before its expiry period.
- Matching must consider quantity, location, urgency, and availability.
- User access must depend on assigned roles.
- Donation and delivery information must remain consistent.
- Transactions must be recorded accurately.
- UML diagrams must be mutually consistent.

15. Requirement-to-Use Case/Class Traceability

Requirement	Use Case	Related Classes
Register surplus food	Register Surplus Food	FoodProvider, FoodDonation
Store food details	Manage Donation	FoodDonation, FoodItem
Find suitable recipient	Match Donation	FoodDonation, FoodRequirement, RecipientOrganization
Request donation	Request Food	RecipientOrganization, DonationRequest
Assign delivery	Manage Delivery	DeliveryPartner, Delivery
Track delivery	Update Delivery Status	Delivery, DeliveryPartner
Monitor expiry	Monitor Expiry	FoodDonation, Notification
Notify users	Send Notification	Notification, User
Manage users	Manage Users	User, Administrator
Generate reports	Generate Reports	Report, Administrator

16. Implementation/Prototype and Results

A prototype of the IFWRRS can be developed as a web or mobile-based application.

Prototype Modules

1. User Registration and Login
2. Food Provider Dashboard
3. Surplus Food Registration
4. Recipient Organization Dashboard
5. Food Matching Module
6. Donation Request Module
7. Delivery Management
8. Expiry Notification

9. Administrator Dashboard

10. Reports

Expected Results

The prototype should demonstrate that:

- Food providers can register surplus food successfully.
- Recipient organizations can view and request available food.
- The system can identify suitable recipients.
- Delivery partners can receive and update delivery assignments.
- Expiry notifications can be generated.
- Administrators can monitor system activities.
- Food donation and delivery status can be tracked.

17. Reflection on Design Decisions and Learning Outcomes

The development of the IFWRRS model helped in understanding how Object-Oriented Analysis and Design concepts can be applied to a real-world problem. The system was analysed by identifying actors, objects, classes, attributes, and methods.

The development of different UML diagrams helped in understanding the structural and behavioural aspects of the system. The Use Case Diagram represents user interactions, while the Class Diagram represents the structure of the system. Sequence, Activity, and State Transition Diagrams describe system behaviour. Package, Component, and Deployment Diagrams represent the architectural structure.

The use of principles such as encapsulation, abstraction, high cohesion, and low coupling improves the maintainability and scalability of the proposed system. Design patterns such as Factory, Observer, and Strategy can further improve reusability and flexibility.

The project is also relevant to **SDG 2 – Zero Hunger, SDG 11 – Sustainable Cities and Communities, SDG 12 – Responsible Consumption and Production, and SDG 13 – Climate Action.**

Overall, the assignment provides practical understanding of converting a real-world problem into an object-oriented software model using UML and applying appropriate design principles to produce a feasible, maintainable, and scalable system.